Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií

FIIT-100241-120954

Andrej Krištof

Úvod do prompt engineeringu

Bakalárska práca: Konceptuálna časť

Študijný program:	Informatika
Študijný odbor:	Informatika
Miesto vypracovania:	Ústav počítačového inžinierstva a aplikovanej informatiky
Vedúci práce:	Ing. Peter Bakonyi, PhD.

Máj 2026

Slovenská technická univerzita v Bratislave
Ústav počítačového inžinierstva a aplikovanej informatiky

Fakulta informatiky a informačných technológií
Akademický rok: 2025/2026
Evidenčné číslo: FIIT-100241-120954

ZADANIE BAKALÁRSKEJ PRÁCE

Študent:	**Andrej Krištof**
ID študenta:	120954
Študijný program:	informatika
Študijný odbor:	informatika
Vedúci práce:	Ing. Peter Bakonyi, PhD.
Vedúci pracoviska:	Ing. Katarína Jelemenská, PhD.

Názov práce: **Úvod do prompt engineeringu**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

V dnešnej dobe sa neustále rozširuje využívanie jazykových modelov umelej inteligencie, ktoré nám uľahčujú monotónnu každodennú prácu, ale sú aj komplexné na správne používanie a obnášajú isté riziká. Analyzujte, navrhnite a vytvorte komplexný návod k prompt engineeringu, ktorý zahŕňa detailný prehľad existujúcich metód a nástrojov v tejto oblasti. Venujte pozornosť krokom a postupom pre efektívne inžinierstvo prompt systémov, vrátane zbierania a predspracovania dát, výberu a trénovania modelov a ich následnej evaluácie. Rovnako zdôraznite etické zásady a bezpečnostné opatrenia, ktoré by mali byť zahrnuté pri implementácii prompt engineeringu. Taktiež zvážte aspekty transparentnosti a vysvetliteľnosti modelov s cieľom uľahčiť porozumenie procesu používateľom. Nakoniec poskytnite príklady a praktické rady, ktoré umožnia čitateľom úspešne implementovať a optimalizovať svoje vlastné prompt systémy.

Rozsah práce: 40

Termín odovzdania bakalárskej práce:	11. 05. 2026
Dátum schválenia zadania bakalárskej práce:	28. 04. 2026
Zadanie bakalárskej práce schválil:	doc. Ing. Ján Lang, PhD. – garant študijného programu

Vyhlásenie

Čestne vyhlasujem, že som túto prácu vypracoval samostatne, na základe konzultácií a s použitím uvedenej literatúry. Za konečný obsah tejto práce preberám plnú zodpovednosť.

V Bratislave, Máj 2026 Andrej Krištof

Poďakovanie

Chcem sa poďakovať vedúcemu svojej bakalárskej práce Ing. Petrovi Bakonyimu, PhD., za jeho rady, pripomienky a trpezlivosť, ktoré mi výrazne pomohli pri vypracovaní tejto práce. Ďalej ďakujem svojej rodine a priateľom za podporu, ktorú mi poskytovali počas celého štúdia.

Anotácia

Slovenská technická univerzita v Bratislave
FAKULTA INFORMATIKY A INFORMAČNÝCH TECHNOLÓGIÍ
Študijný program: Informatika

Autor: Andrej Krištof
Bakalárska práca: Úvod do prompt engineeringu
Vedúci bakalárskej práce: Ing. Peter Bakonyi, PhD.
Máj 2026

Táto bakalárska práca sa zaoberá optimalizáciou promptov pre veľké jazykové modely (LLM). Cieľom je otestovať a porovnať dvanásť promptovacích techník: od základných (Zero-Shot, Few-Shot, Chain-of-Thought, Role-Playing) cez pokročilé (Self-Consistency, Generated Knowledge, Directional Stimulus, Tree-of-Thoughts, Step-Back) až po vlastné kombinované techniky (Few-Shot Self-Consistency, Critique and Refine, Role-Playing Few-Shot). Experimenty prebiehali na siedmich open-source modeloch (Phi-3, Minstral-3, Qwen2.5, Llama 3.2, DeepSeek R1, Qwen3, Gemma3) spustených lokálne cez Ollama. Testovanie sa zameralo na štyri typy úloh: sumarizáciu, generovanie receptu, tvorbu LinkedIn príspevkov a brainstorming. Pre každú kombináciu boli merané tokeny, latencia a kvalita výstupu, pričom kvalita bola hodnotená porovnaním s referenčnými benchmarkmi pomocou evaluačného modelu gpt-oss:20b. Hlavným zistením je, že Few-Shot je najuniverzálnejšou technikou, zatiaľ čo zložité techniky prinášajú horšie výsledky pri vyšších nákladoch. Výstupom práce je praktická príručka prompt engineeringu pre široké spektrum používateľov. Práca prináša systematické porovnanie overené tromi nezávislými experimentmi, čím poskytuje spoľahlivý podklad pre efektívnu komunikáciu s LLM v praxi.

Annotation

Slovak University of Technology Bratislava
FACULTY OF INFORMATICS AND INFORMATION TECHNOLOGIES
Degree course: Informatics

Author: Andrej Krištof
Bachelor's Thesis: Introduction to prompt engineering
Supervisor: Ing. Peter Bakonyi, PhD.
2026, May

This bachelor's thesis focuses on prompt optimization for large language models (LLMs). The aim is to test and compare twelve prompting techniques: from basic (Zero-Shot, Few-Shot, Chain-of-Thought, Role-Playing) through advanced (Self-Consistency, Generated Knowledge, Directional Stimulus, Tree-of-Thoughts, Step-Back) to custom combined techniques (Few-Shot Self-Consistency, Critique and Refine, Role-Playing Few-Shot). Experiments were conducted on seven open-source models (Phi-3, Minstral-3, Qwen2.5, Llama 3.2, DeepSeek R1, Qwen3, Gemma3) running locally via Ollama. Testing focused on four tasks: summarization, recipe generation, LinkedIn post creation, and brainstorming. For each combination, tokens, latency, and output quality were measured, where quality was evaluated by comparison with reference benchmarks using the gpt-oss:20b evaluation model. The main finding is that Few-Shot is the most universal technique, while complex techniques yield worse results at higher costs. The outcome is a practical prompt engineering guide for a wide range of users. The thesis provides a systematic comparison validated by three independent experiments, offering a reliable basis for effective real-world communication with LLMs.

Obsah konceptuálnej časti

Technický abstrakt **1**

Laický súhrn **2**

1 Úvod **3**

2 Definovanie problému a riešenie **5**

2.1 Definovanie problému 5

2.1.1 Dôsledky neexistencie overeného porovnania 6

2.1.2 Navrhované riešenie 6

2.2 Prehľad podobných riešení 7

2.2.1 Základné metódy prompt engineeringu 7

2.2.2 Pokročilé techniky pre zlepšenie uvažovania 7

2.2.3 Automatizované prístupy k optimalizácii 8

2.2.4 Špecializované aplikácie a metódy 9

2.2.5 Etické a bezpečnostné aspekty 9

2.2.6 Zhrnutie 10

2.3 Prehľad riešenia (vysoká úroveň) 10

2.3.1 Návrh testovacieho systému 10

2.3.2 Návrh evaluačného skriptu 11

2.3.3 Výber optimalizačných techník 12

2.3.4 Opodstatnenie výberu techník 13

2.3.5 Výber testovacích modelov 14
2.3.6 Obmedzenia riešenia 14
2.3.7 Návrh príručky pre prompt engineering 14
2.3.8 Interdisciplinárny dosah 15
2.4 Posúdenie rizík 15
2.4.1 Silné stránky 16
2.4.2 Slabé stránky a stratégie zmiernenia 16
2.4.3 Hrozby a stratégie zmiernenia 16
2.5 Experimentálna reproducibilita a integrácia 17
2.5.1 Architektúra experimentálneho prostredia 17
2.5.2 Evaluačný skript 18
2.5.3 Konfigurácia a dokumentácia 18
2.5.4 Reprodukovateľný testovací proces 19
2.5.5 Verejná dostupnosť 20
2.6 Udržateľnosť a vplyv na životné prostredie 20
2.6.1 Technická udržateľnosť 21
2.6.2 Dokumentácia a archivácia 21
2.6.3 Environmentálna udržateľnosť 21
2.6.4 Strategia rozšírenia 22
2.7 Zamestnateľnosť 22
2.8 Teamwork, diverzita a inklúzia 23

3 Záver **25**
3.1 Osobný odborný rozvoj a získané poznatky 25
3.2 Prínos pre prax a používateľov 26
3.3 Širší význam a zhrnutie 26

Zoznam skratiek **27**

Zoznam obrázkov **28**

Literatúra **29**

A Príručka na prompt engineering **A-1**

B Popis digitálneho odovzdania **B-1**

C Technická dokumentácia **C-1**

C.1 Nastavenie C-1

C.2 Nastavenie evaluačného prostredia C-2

C.3 Štruktúra projektu C-2

C.4 Návod na inštaláciu C-3

C.5 Používateľská príručka C-4

C.5.1 Spustenie testovacieho systému C-4

C.5.2 Spustenie evaluácie C-4

C.5.3 Štruktúra výstupov C-5

C.5.4 Konfigurácia C-5

C.6 Kľúčové časti implementácie C-6

C.6.1 Hlavný testovací skript (tester.py) C-6

C.6.2 Evaluačný skript (evaluation.py) C-7

C.6.3 Analytický skript (analysis.py) C-8

D Harmonogram práce **D-1**

D.1 Plán práce pre BP1 . . . . . D-1

D.2 Plán práce pre BP2 . . . . . D-2

E Použitie umelej inteligencie **E-1**

E.1 Opatrenia číslo 1/2024 - O E-1

Technický abstrakt

Veľké jazykové modely sa stali kľúčovým nástrojom, no ich efektívne využitie závisí od kvality promptov. Táto práca prináša systematické porovnanie dvanástich promptovacích techník na siedmich menších open-source modeloch (1.5-4B parametrov), čím prispieva k málo preskúmanej oblasti prompt engineeringu pre kompaktné LLM. Cieľom práce je otestovať a porovnať dvanásť rôznych promptovacích techník, od základných až po vlastné kombinované prístupy, a na základe experimentálnych výsledkov vytvoriť praktickú príručku pre efektívnu komunikáciu s LLM.

Konceptuálna časť práce zahŕňa detailný prehľad existujúcich metód prompt engineeringu, analýzu etických a bezpečnostných aspektov a návrh testovacej metodológie. Vedecká časť je zameraná na porovnanie uvedených techník naprieč siedmimi open-source modelmi s veľkosťou 1.5-4B parametrov, spustenými lokálne prostredníctvom platformy Ollama. Testovanie prebiehalo na štyroch typoch úloh: sumarizácia textu, generovanie receptu, tvorba LinkedIn príspevkov a brainstorming nápadov. Pre každú kombináciu techniky, modelu a úlohy boli merané metriky ako počet tokenov, doba odozvy a kvalita výstupu. Kvalita bola hodnotená porovnaním s referenčnými benchmarkmi pomocou evaluačného modelu gpt-oss:20b. Hlavným zistením je, že technika Few-Shot sa ukázala ako najuniverzálnejšia naprieč všetkými modelmi a úlohami.

Výsledkom práce je praktická príručka prompt engineeringu so základnými princípmi, optimalizačnými technikami a príkladmi. Navrhnutý postup zavádza systém do praxe prompt engineeringu a zefektívňuje využívanie LLM.

Laický súhrn

Bakalárska práca sa zaoberá tým, ako ľudia píšu otázky a príkazy pre umelú inteligenciu, ako je ChatGPT, a ako spôsob zadania týchto otázok ovplyvňuje kvalitu odpovede. Takéto zadania sa nazývajú prompty a ich správna formulácia je dnes veľmi dôležitá, nielen pre programátorov, ale aj pre bežných používateľov.

Cieľom mojej práce je otestovať rôzne spôsoby písania promptov a zistiť, ktoré fungujú najlepšie. V prvej, konceptuálnej časti som zmapoval existujúce metódy a techniky, analyzoval som bezpečnostné riziká spojené s používaním umelej inteligencie a navrhol som spôsob, ako budem prompty testovať.

Vo vedeckej časti skúšam rôzne spôsoby zadávania otázok na siedmich rôznych modeloch umelej inteligencie, ktoré bežia na mojom počítači. Meriam, ako rýchlo reagujú, koľko „tokenov“ (jednotiek textu) použijú a ako sa mení kvalita ich odpovedí. Všetky odpovede som porovnal s referenčnými vzormi, aby som vedel objektívne zhodnotiť, ktoré techniky sú najlepšie. Na základe týchto poznatkov som vytvoril praktický návod, ktorý pomôže ľuďom písať otázky tak, aby umelá inteligencia dávala zrozumiteľné a presné odpovede bez zbytočného plytvania výpočtovým časom.

Výsledkom je univerzálny súbor overených odporúčaní, ktorý je použiteľný pre široké spektrum používateľov od technicky zdatných až po tých, ktorí umelú inteligenciu využívajú len príležitostne. Práca tak pomáha lepšie pochopiť, ako komunikovať s umelou inteligenciou efektívnejšie a ako z nej dostať čo najlepšie výsledky.

Kapitola 1

Úvod

V posledných rokoch zažívame výrazný posun v tom, ako ľudia komunikujú s technológiami. Zavedenie veľkých jazykových modelov (Large Language Models - LLMs) predstavuje „paradigm shift, ktorý umožňuje generovanie textu často nerozoznateľného od textu napísaného človekom“ [1]. Modely ako GPT, LLaMA či Mistral dokážu spracovať obrovské množstvo informácií a generovať texty porovnateľné s ľudským prejavom. Ako poukazujú Chen a kol., tieto modely „zlepšili úlohy od extrakcie informácií až po vytváranie pútavého obsahu“ [2]. Ich úspech však vo veľkej miere závisí od spôsobu, akým s nimi používateľ komunikuje.

Preto prompt engineering predstavuje spôsob, akým používateľ efektívne komunikuje s umelou inteligenciou. Namiesto zložitých úprav modelu sa výstup ovplyvňuje vhodne formulovaným textom. Ako uvádzajú Sahoo a kol., „prompt engineering sa stal nevyhnutnou technikou rozširovania schopností veľkých jazykových modelov bez potreby úpravy ich vnútorných parametrov“ [3]. Dobre navrhnutý prompt dokáže výrazne zlepšiť kvalitu výstupu, zatiaľ čo nejasný alebo neštruktúrovaný prompt môže viesť k nepresným alebo irelevantným od-

povediam [4].

Základné techniky prompt engineeringu zahŕňajú poskytovanie jasných inštrukcií, role prompting či few-shot príklady [2]. Pokročilé metódy ako Chain-of-Thought, Self-Consistency či Tree-of-Thoughts nútia model explicitne zapisovať svoje uvažovanie, čo zlepšuje riešenie zložitých logických problémov [3]. V oblasti automatizácie sa objavujú prístupy ako APO-CF [5] alebo SPRIG [6]. Na základe prehľadu existujúcich prác sa táto práca zameriava na porovnanie vybraných techník prompt engineeringu na menších open-source modeloch (1.5-4B parametrov) naprieč rôznorodými úlohami, ako je sumarizácia, generovanie receptov, písanie LinkedIn príspevkov a brainstorming.

Cieľom tejto práce je preto analyzovať vplyv rôznych techník prompt engineeringu na kvalitu výstupov veľkých jazykových modelov. Zameriavam sa na porovnanie dvanástich prístupov na siedmich open-source modeloch spustených lokálne cez Ollama. Výsledkom práce je porovnanie jednotlivých techník a praktická príručka pre efektívne využívanie LLM.

Kapitola 2

Definovanie problému a riešenie

2.1 Definovanie problému

Hlavným problémom, ktorý táto bakalárska práca rieši, je neexistencia experimentálne overeného porovnania promptovacích techník na menších open-source modeloch, ktoré by používateľom poskytlo jasné a praktické odporúčania pre efektívnu komunikáciu s LLM. Napriek rastúcemu záujmu o Small Language Models (SLM) stále chýbajú štandardizované benchmarky a systematické evaluačné metodiky pre porovnanie promptovacích techník naprieč rôznymi úlohami a modelmi [7].

Cieľom tejto práce je preklenúť túto medzeru vytvorením overeného postupu pre prompt engineering, ktorý bude založený na experimentálnych dôkazoch, všeobecne použiteľný bez ohľadu na úroveň skúseností používateľa a prakticky aplikovateľný v reálnych scenároch.

2.1.1 Dôsledky neexistencie overeného porovnania

Absencia systematického porovnania promptovacích techník vedie k viacerým problémom. Používatelia často volia techniky intuitívne bez znalosti ich vhodnosti, čo vedie k nekonzistentnej kvalite výstupov. Príliš krátky prompt neposkytuje dostatočný kontext, zatiaľ čo príliš dlhý môže zavádzať irelevantné informácie a zhoršovať výkon [4]. Nevhodne zvolené techniky zároveň generujú zbytočne dlhé výstupy, čo zvyšuje spotrebu tokenov a latenciu, a teda aj finančné náklady pri komerčnom využití API. Chýbajúce overené postupy môžu viesť k bezpečnostným a etickým rizikám, ako je prompt injection alebo jailbreaking [2]. Nízka prehľadnosť napokon bráni používateľom pochopiť, prečo niektoré techniky fungujú lepšie, pričom [6] ukázali, že systémové prompty výrazne menia „myšlienkový priestor“ modelu, tento poznatok je využiteľný len vtedy, ak existuje porovnanie rôznych techník.

2.1.2 Navrhované riešenie

Na základe identifikovaného problému navrhujem experimentálne riešenie pozostávajúce z troch hlavných častí. Prvou časťou je implementácia automatizovaného testovacieho systému, ktorý umožní systematické porovnanie dvanástich promptovacích techník na siedmich open-source modeloch a štyroch typoch úloh (sumarizácia, recepty, LinkedIn príspevky, brainstorming). Nasleduje evaluácia výstupov pomocou referenčných benchmarkov a externého modelu gpt-oss:20b, kde sa meria kvalita, tokenová efektivita a latencia. Posledná časť je vytvorenie praktickej príručky, ktorá sumarizuje overené odporúčania a ktorá technika je najvhodnejšia pre aký typ úlohy.

2.2 Prehľad podobných riešení

Existujúce práce možno rozdeliť do niekoľkých kategórií podľa prístupu k optimalizácii promptov.

2.2.1 Základné metódy prompt engineeringu

Základné techniky prompt engineeringu zahŕňajú poskytovanie jasných inštrukcií, role prompting, používanie oddelovačov či one-shot a few-shot príklady [2]. Tripathi a kol. zdôrazňujú, že kľúčové je poskytnúť modelu dostatočný kontext a rozčleniť komplexné úlohy na menšie kroky, pričom upozorňujú na nelineárny vzťah medzi komplexnosťou promptu a výkonom modelu, to znamená, že príliš krátky prompt neposkytne dostatok kontextu a príliš dlhý zavádza irelevantné informácie a zhoršuje výkon [4]. Aj keď sú tieto metódy ľahko použiteľné, ich hlavným obmedzením je, že nefungujú rovnako dobre na zložitých logických alebo viacstupňových úlohách, kde model potrebuje explicitné vedenie.

2.2.2 Pokročilé techniky pre zlepšenie uvažovania

Podľa [3] boli pre zlepšenie logického uvažovania vyvinuté techniky ako Chain-of-Thought (CoT), Self-Consistency, Tree-of-Thoughts (ToT) a Self-Refine Prompting. CoT nútí model zapisovať si postupné uvažovanie, čo výrazne zlepšuje riešenie zložitých problémov, ale zvyšuje počet tokenov a výpočtové náklady. Self-Consistency generuje viacero myšlienkových reťazcov a vyberá najčastejšiu odpoveď, čím zvyšuje spoľahlivosť za cenu viacnásobného volania modelu. ToT rozkladá problém na strom možností a hodnotí jednotlivé vetvy, čo je ešte výpočtovo náročnejšie.

Cheng a kol. predstavujú Black-Box Prompt Optimization (BPO), ktorý optima-

lizuje prompty bez prístupu k vnútorným parametrom modelu, čo je cenné pre uzavreté modely ako GPT-4 [8]. BPO je univerzálny naprieč modelmi a úlohami, jeho obmedzením je však závislosť na kvalite tréningových dát a vyššia počiatočná réžia pri tvorbe optimalizátora.

2.2.3 Automatizované prístupy k optimalizácii

Automatizácia prompt engineeringu prináša metódy ako APO-CF, ktorá využíva confusion matrix feedback na jednokrokovú úpravu promptu, čím dramaticky znižuje výpočtovú náročnosť oproti iteračným postupom [5]. Jej obmedzením je závislosť od konkrétneho modelu pretože výborne fungovala na GPT-4o, ale nie na DeepSeek-Chat.

Metóda SPRIG používa genetický algoritmus na optimalizáciu systémových promptov, pričom vytvára univerzálne inštrukcie použiteľné naprieč úlohami [6]. Hoci je efektívna, optimalizovaný systémový prompt nemusí byť optimálny pre špecifické úlohy a vyžaduje starostlivú inicializáciu komponentov.

AutoPDL rozširuje automatizáciu na LLM agentov a optimalizuje nielen obsah promptov, ale aj výber vhodných agentických metód (ReAct, ReWOO) [9]. Výrazne zlepšuje presnosť (až o 67,5 percentuálnych bodov), no jeho výsledky sú často modelovo-špecifické a nie sú prenositeľné medzi rôznymi architektúrami. Li a kol. systematicky kategorizujú automatické metódy do štyroch skupín: optimalizácia pomocou veľkých modelov, evolučné metódy, gradientové prístupy a reinforcement learning [10]. Ich prehľad ukazuje, že žiadna z týchto metód nie je univerzálne najlepšia a výber závisí od konkrétneho modelu a úlohy.

2.2.4 Špecializované aplikácie a metódy

V špecializovaných oblastiach Yamada skúma optimalizáciu promptov pre strojový preklad, pričom ukazuje, že špecifikácia účelu a cieľového publika výrazne zlepšuje kvalitu výstupov [11]. Sabbatella a kol. analyzujú prompt engineering ako kombinatorický problém a navrhujú bayesovskú optimalizáciu, ktorá je vzorkovo efektívna, ale vyžaduje vhodné nastavenie priestoru [12]. Bystroński a kol. predstavujú LatentPrompt, ktorý optimalizuje prompty v latentnom priestore pomocou interpolácie, čím generuje čitateľné inštrukcie bez prístupu k gradientom [13]. Táto metóda je však výpočtovo náročná a vyžaduje kvalitný enkodér.

2.2.5 Etické a bezpečnostné aspekty

Chen a kol. upozorňujú na prompt injection a jailbreaking ako kľúčové bezpečnostné hrozby a navrhujú viacvrstvové obranné stratégie vrátane validácie promptov a moderácie výstupov [2]. Kumar a kol. systematicky analyzujú bezpečnostné riziká a predstavujú klasifikačný systém pre prompty so šablónovanými odpoveďami a pravidelným monitoringom [14]. Ich prístup je praktický, ale vyžaduje neustálu aktualizáciu šablón podľa nových typov útokov.

V oblasti transparentnosti Feng a kol. prispievajú metódou PromptExplainer, ktorá využíva MLM hlavy na identifikáciu kľúčových slov v promptoch, čo zvyšuje vysvetliteľnosť rozhodnutí modelu [15]. Metóda je však navrhnutá pre modely typu BERT a nie je priamo použiteľná pre generatívne modely ako GPT. Tripathi a kol. zdôrazňujú potrebu metodologickej prísnosti pri používaní LLM v citlivých oblastiach a Gao navrhuje postupy pre vývoj robustných promptovacích techník odolných voči zneužitiu [16].

2.2.6 Zhrnutie

Z uvedeného prehľadu vyplýva, že väčšina existujúcich prác sa zameriava na veľké komerčné modely (GPT-4, Claude) alebo na automatizovanú optimalizáciu promptov. Chýba systematické experimentálne porovnanie manuálnych promptovacích techník na menších open-source modeloch (1.5-4B parametrov), ktoré sú dostupné pre širokú verejnosť a možno ich spúšťať lokálne bez obmedzení. Táto práca preto prináša takéto porovnanie na siedmich modeloch a štyroch úlohách, pričom meria nielen kvalitu výstupov, ale aj tokenovú a časovú efektivitu, a poskytuje praktické odporúčania vo forme príručky.

2.3 Prehľad riešenia (vysoká úroveň)

Na riešenie identifikovaných problémov v oblasti prompt engineeringu bolo navrhnuté experimentálne testovanie optimalizačných techník. Hlavným prístupom je porovnanie efektívnosti rôznych metód prompt engineeringu na špecifikovaných use cases pomocou automatizovaného testovacieho programu v jazyku Python.

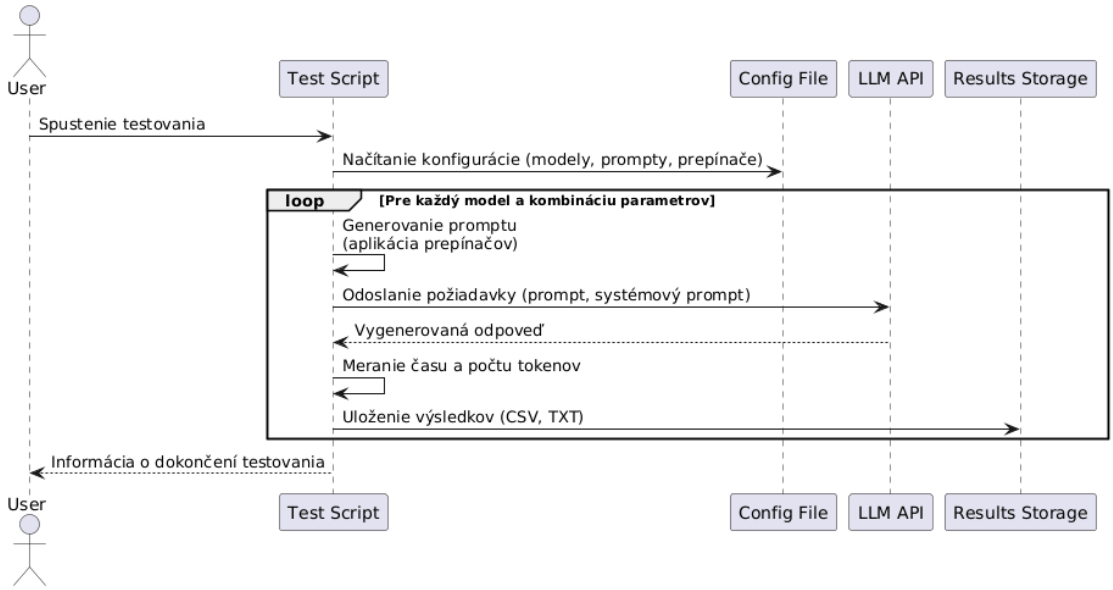
2.3.1 Návrh testovacieho systému

Pre experimentálne overenie bol vyvinutý automatizovaný testovací systém, ktorý umožňuje komplexné hodnotenie rôznych promptovacích techník. Systém je implementovaný v jazyku Python a pozostáva z nasledujúcich komponentov:

- **Konfiguračný systém** - JSON súbor definujúci use cases, prompt techniky a parametre
- **API komunikácia** - integrácia s Ollama lokálnymi modelmi cez REST API

- **Kombinatorické testovanie** - generovanie všetkých kombinácií parametrov
- **Meranie metrík** - sledovanie tokenov, latencie a kvality výstupu

Ako je znázornené na Obr. 2.1, proces testovania pozostáva z viacerých krokov.



Obr. 2.1: Sekvenčný diagram procesu testovania jazykových modelov

2.3.2 Návrh evaluačného skriptu

Na objektívne vyhodnotenie kvality výstupov bol vyvinutý samostatný evaluačný skript, ktorý používa externý LLM model (gpt-oss:20b) ako automatizovaného hodnotiteľa. Na spracovanie nameraných dát a generovanie prehľadných agregácií (priemerné skóre podľa techniky, modelu alebo úlohy) bol vytvorený pomocný skript `analysis.py`. Evaluačný skript najprv načíta výstupy z testovacieho systému, extrahuje potrebné údaje (použitý model, typ úlohy, promptovaciu techniku, počet tokenov, latenciu a kompletný výstup).

Pre každý test skript dynamicky vytvorí evaluačný prompt obsahujúci referenčný benchmark, namerané hodnoty a výstup modelu. Evaluačný model ohodnotí výstup na škále 1-10 v troch kategóriách: kvalita výstupu, tokenová efektivita a latenčná efektivita, následne vypočíta celkové skóre. Aby sa zabezpečila konzistentnosť, evaluačný model vracia výlučne validný JSON formát.

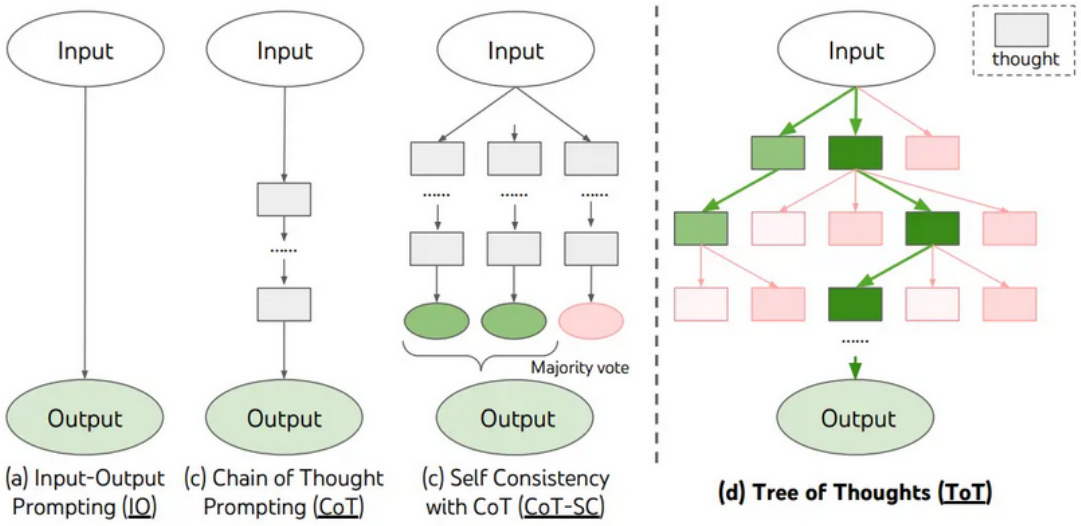
Spracované odpovede sú uložené do CSV súboru pre ďalšiu analýzu a surové JSON odpovede do textového logu pre prípadnú manuálnu kontrolu.

2.3.3 Výber optimalizačných techník

Pre experimentálne overenie bolo vybraných 12 optimalizačných techník prompt engineeringu (kde posledné 3 sú moje), ktoré pokrývajú široké spektrum prístupov od základných po pokročilé. **Zero-Shot Prompting** je základná metóda bez príkladov, ktorá podľa [17] sama o sebe nestačí na zložité logické úlohy (napr. MultiArith 17,7%). **Few-Shot Prompting** používa ukážkové príklady [18]. **Chain-of-Thought (CoT)** predstavuje premýšľanie krok za krokom [19]. **Role-Playing** priraďuje modelu určenú rolu [20]. **Self-Consistency** generuje množstvo rôznych uvažovacích ciest a vyberá odpoveď s najvyššou zhodou medzi nimi [21]. **Generated Knowledge** generuje dodatočné znalostné vety k otázke a použije ich ako rozšírený vstup pre zodpovedanie otázok [22]. **Directional Stimulus** smeruje pozornosť modelu na špecifické aspekty úlohy [23]. **Tree-of-Thoughts** umožňuje modelu zvažovať viaceré perspektívy alebo prístupy k riešeniu úlohy [24]. **Step-Back Prompting** najprv identifikuje všeobecné princípy danej úlohy a až potom ich aplikuje na konkrétny problém [25]. **Few-Shot Self-Consistency** je kombinácia Few-Shot (ukážkové príklady) a Self-Consistency (generovanie viacerých verzií a výber najlepšej). **Critique and Refine** je trojfázová technika, pri ktorej model najprv vygeneruje počiatočnú odpoveď, následne ju kriticky zhod-

notí z hľadiska presnosti, úplnosti a jasnosti, a nakoniec na základe tejto kritiky vytvorí vylepšenú verziu. **Role-Playing Few-Shot** kombinuje Role-Playing (špecifická rola) a Few-Shot (ukážkové príklady).

Na Obr. 2.2, prevzatom z [24], je možné vidieť schematické znázornenie rôznych prístupov k riešeniu problémov s LLM. Každý obdĺžnik predstavuje „myšlienku“, čo je logicky prepojená jazyková sekvencia slúžiaca ako medzikrok pri riešení problému.



Obr. 2.2: Ilustrácia Tree of Thoughts

2.3.4 Opodstatnenie výberu techník

Výber techník pokrýva rôzne úrovne komplexnosti. Základné metódy (Zero-Shot, Few-Shot, Chain-of-Thought, Role-Playing) slúžia ako benchmark, zatiaľ čo pokročilé techniky (Self-Consistency, Generated Knowledge, Directional Stimulus, Tree-of-Thoughts, Step-Back) patria medzi často skúmané prístupy v odbornej literatúre. Štyri typy úloh (sumarizácia, recepty, LinkedIn príspevky, brainstorming) umožňujú overiť univerzálnosť techník naprieč rôznymi domé-

nami. Tri vlastné techniky (Few-Shot Self-Consistency, Critique and Refine, Role-Playing Few-Shot) boli navrhnuté s predpokladom, že kombinácia osvedčených prístupov môže priniesť synergický efekt.

2.3.5 Výber testovacích modelov

Pre experimentálne testovanie bolo vybraných 7 rôznych LLM modelov integrovaných cez platformu Ollama, ktoré boli použité rovnako pre všetky testovacie scenáre. Výber zahŕňa **phi3:3.8b** (Microsoft) [26], **ministral-3:3b** (Mistral AI) [27], **qwen2.5:3b** (Alibaba) [28], **llama3.2:3b** (Meta) [29], **deepseek-r1:1.5b** (distilovaný model) [30], **qwen3:4b** (Alibaba) [31] a **gemma3:4b** (Google) [32]. Modely sa líšia veľkosťou (1.5B - 4B parametrov), architektúrou a vývojármi, čo umožňuje testovať univerzálnosť promptovacích techník naprieč rôznymi typmi modelov.

2.3.6 Obmedzenia riešenia

Hoci navrhnutý testovací systém poskytuje systematické porovnanie, výsledky sú limitované na modely do 4B parametrov a nemusia priamo preniesť na väčšie modely (napr. GPT-4). Taktiež evaluácia pomocou LLM modelu (gpt-oss:20b) prináša určitú mieru subjektivity, ktorá bola minimalizovaná použitím teploty 0 a prísneho systémového promptu.

2.3.7 Návrh príručky pre prompt engineering

Súčasťou riešenia je praktická príručka pre prompt engineering založená na výsledkoch experimentov. Je rozdelená do štyroch častí: základné princípy, optimalizačné techniky, praktické príklady a etické a bezpečnostné odporúčania. Základné princípy zahŕňajú jasnosť inštrukcií, kontextualizáciu, formá-

tovanie a špecifikáciu role, každý s konkrétnymi príkladmi. Druhá časť popisuje dvanásť optimalizačných techník: základné (Zero-Shot, Few-Shot, Chain-of-Thought, Role-Playing), pokročilé (Self-Consistency, Generated Knowledge, Directional Stimulus, Tree-of-Thoughts, Step-Back) a vlastné kombinované (Few-Shot Self-Consistency, Critique and Refine, Role-Playing Few-Shot). Pre každú je uvedené, kedy ju použiť, a konkrétny príklad. Tretia časť obsahuje štyri praktické ukážky (sumarizácia, recepty, LinkedIn príspevky, brainstorming) s porovnaním techník a odporúčaniami. Štvrtá časť obsahuje etické a bezpečnostné odporúčania pre zodpovedné používanie LLM, vrátane zásad ochrany osobných údajov a prevencie zneužitia. Kompletná príručka je súčasťou prílohy.

2.3.8 Interdisciplinárny dosah

Výsledky práce majú interdisciplinárny dosah, môžu byť využité v pedagogike (výučba efektívnej komunikácie s AI), marketingu (tvorba LinkedIn príspevkov), psychológii (štúdium človek-AI interakcie) a v biznisovej praxi (optimalizácia promptov pre zákaznícke chatboty).

2.4 Posúdenie rizík

V rámci práce zameranej na výskum a optimalizáciu promptov pre LLM bola vykonaná základná SWOT analýza, ktorá identifikuje silné a slabé stránky projektu, a hrozby. Zároveň sú navrhnuté spôsoby na zmiernenie identifikovaných rizík.

2.4.1 Silné stránky

Práca sa opiera o modernú a rýchlo sa rozvíjajúcu oblasť umelej inteligencie, čo zvyšuje jej aktuálnosť a vedeckú hodnotu. Použitie otvorených modelov prostredníctvom platformy Ollama umožňuje transparentnosť, reprodukovateľnosť a experimentovanie bez komerčných obmedzení. Presne určená metodika testovania promptov podporuje jednotný a logický postup aj objektívne posudzovanie výsledkov.

2.4.2 Slabé stránky a stratégie zmiernenia

Medzi slabé stránky patrí rýchla evolúcia LLM technológií. Nové verzie modelov môžu meniť správanie systému a ovplyvniť porovnateľnosť výsledkov. Toto riziko je zmiernené zameraním sa na promptovacie techniky, ktoré nie sú viazané na konkrétnu verziu modelu. Ďalšou slabinou je obmedzený prístup k výkonným hardvérovým zdrojom, ktorý môže limitovať rozsah experimentov. Tomu sa predchádza použitím menších modelov (1.5-4B parametrov), postupným spúšťaním experimentov a využitím samostatného výkonnejšieho počítača pre evaluáciu. Subjektívne hodnotenie kvality odpovedí predstavuje potenciálny zdroj nepresností. Tento problém je riešený automatizovaným evaluačným skriptom s modelom gpt-oss:20b pri teplote 0, ktorý poskytuje konzistentné a reprodukovateľné hodnotenie.

2.4.3 Hrozby a stratégie zmiernenia

Medzi hlavné hrozby patrí nepredvídateľné správanie LLM, ktoré môže viesť k ťažkostiam pri reprodukovaní výsledkov. Tomu sa predchádza opakovaním experimentov v troch nezávislých meraniach a meraním štatistickej stability výsledkov. Ďalšou hrozbou je možné zneužitie príručky a optimalizačných techník

na účely jailbreakingu alebo prompt injection útokov. Toto riziko je zmiernené zahrnutím etických odporúčaní priamo do príručky a zdôraznením zodpovedného používania. Efektívnejšie promptovacie techniky môžu zvýšiť efektivitu generovania škodlivého obsahu, čo predstavuje etické riziko. Tomuto sa predchádza transparentným publikovaním výsledkov a odporúčaním používať techniky výlučne na legálne a prosociálne účely.

2.5 Experimentálna reproducibilita a integrácia

Pre zabezpečenie reprodukovateľnosti a transparentnosti výskumu boli implementované opatrenia, ktoré umožňujú presnú replikáciu experimentov a overenie získaných výsledkov.

2.5.1 Architektúra experimentálneho prostredia

Experimentálne prostredie je tvorené Python testovacím systémom (popísaný v 2.3), ktorý pozostáva z troch hlavných komponentov zabezpečujúcich konzistentnosť a reprodukovateľnosť. Konfiguračný systém založený na JSON formáte centralizuje všetky experimentálne parametre na jednom mieste. Eliminuje tak nekonzistencie medzi rôznymi behmi a umožňuje úpravy bez zásahu do zdrojového kódu. Testovací skript je štruktúrovaný do logických, okomentovaných sekcií, ktoré pokrývajú načítanie konfigurácie, kombinatorické generovanie testov, komunikáciu s Ollama API a export výsledkov. Manažment závislostí prostredníctvom súboru requirements.txt fixuje verzie všetkých použitých knižníc, čo umožňuje reprodukciu experimentov s identickým softvérovým základom aj pri budúcich aktualizáciách. Kontajnerizácia (napr. Docker) nebola použitá, všetky závislosti sú riešené cez requirements.txt a platforma Ollama sama poskytuje reprodukovateľné prostredie pre spúšťanie jazykových modelov.

2.5.2 Evaluačný skript

Na objektívne a reprodukovateľné vyhodnotenie výstupov bol vyvinutý samostatný evaluačný skript využívajúci externý LLM model (gpt-oss:20b) ako automatizovaného hodnotiteľa, čím sa eliminuje subjektivita ľudského posudzovania. Skript prechádza výstupy z testovacieho systému uložené v full_outputs.txt, pre každý test extrahuje identifikátor, model, úlohu, techniku, počet tokenov, latenciu a kompletný výstup. Dynamicky vytvorí evaluačný prompt obsahujúci referenčný benchmark a požiada model o ohodnotenie na škále 1-10 v troch kategóriách: kvalita, tokenová efektivita a latenčná efektivita. Evaluačný model vracia výlučne validný JSON, ktorý skript spracuje, validuje a uloží do CSV súboru (evaluation_results.csv). Surové JSON odpovede sú zároveň uložené v textovom logu. Celý proces je plne automatizovaný a reprodukovateľný.

2.5.3 Konfigurácia a dokumentácia

Ako bolo spomenuté v 2.5.1, experimentálne parametre sú centralizované v konfiguračnom súbore config.json. Tento súbor je štruktúrovaný do logických sekcií: definícia testovaných modelov (zoznam siedmich LLM), štyri typy úloh (sumarizácia, generovanie receptu, LinkedIn príspevky, brainstorming), kompletná sada 12 základných promptov pre každú úlohu, rôzne varianty systémových promptov a sada konfigurovateľných prepínačov (verbosita, formát výstupu, cieľové publikum a pod.). Takto navrhnutá štruktúra zabezpečuje, že celý experimentálny dizajn je explicitne zdokumentovaný v jednom súbore, čo umožňuje jednoduchú modifikáciu parametrov bez úprav kódu a tiež preskúmať vplyv jednotlivých komponentov promptu na výkon modelu, čo zodpovedá princípu metódy PE2, ktorá sa zameriava na „prompt engineering the meta-prompt“ [33].

2.5.4 Reprodukovateľný testovací proces

Testovací proces je plne automatizovaný. Najprv sa načítajú konfiguračné parametre z config.json (modely, úlohy, techniky, prepínače), následne algoritmus kombinatoricky vygeneruje všetky kombinácie parametrov a pre každú dynamicky vytvorí finálny prompt nahradením placeholderu [TEXT] testovacím vstupom. Komunikácia s modelmi prebieha cez Ollama REST API, kde každá požiadavka meria latenciu (hoci závisí od hardvérového zaťaženia). Reprodukovateľnosť je zabezpečená zaznamenávaním nameraných hodnôt, ktoré možno porovnávať medzi rôznymi behmi. Výsledky sa ukladajú do CSV súboru (kvantitatívna analýza) a do textového logu full_outputs.txt (kompletná kvalitatívna kontrola). Ukážka výstupu je na Obr. 2.3. Po ukončení testovania nasleduje automatická evaluácia (sekcia 2.5.2).

```
=====
TEST ID: 6
MODEL: phi3:3.8b
USE CASE: uc1_summary
PROMPT TECHNIQUE: chain_of_thought
SWITCHES:
- temperature: 0.5
SYSTEM PROMPT: You are a critical analyst who prioritizes extracting core
information, removing redundancy, and avoiding stylistic embellishment.
INPUT TOKENS: 442
OUTPUT TOKENS: 159
LATENCY (SECONDS): 4.042
-----
FULL OUTPUT:
1. Horses (Equus caballus), domesticated around 6,000 years ago by Indo-
European tribes in the Eurasian steppes, were essential for transportation
and agriculture before mechanized vehicles became prevalent.

2. The bond between humans and horses has been significant throughout
history, reflected in language with terms like chivalry and cavalier;
culturally important as seen through myths such as that of Bucephalus, the
horse of Alexander the Great.

3. Beyond their historical roles, practical uses for various parts of a
horse's body have been found across Europe and Iceland, while horses
continue to inspire artistic expression from ancient drawings to
contemporary literature.
=====
```

Obr. 2.3: Príklad výstupu v súbore full_outputs.txt

2.5.5 Verejná dostupnosť

Na zabezpečenie reprodukovateľnosti boli všetky komponenty experimentálneho systému sprístupnené vo verejnom Git repozitári na platforme GitHub (link sa vyskytuje v prílohách). Repozitár obsahuje kompletný zdrojový kód testovacieho systému implementovaného v Pythone vrátane konfiguračného súboru vo formáte JSON, ktorý definuje všetky experimentálne parametre: testované modely, use cases, promptovacie techniky, systémové prompty a prepínače. Súčasťou repozitára je tiež evaluačný skript, ktorý automaticky vyhodnocuje kvalitu výstupov pomocou externého LLM modelu a generuje štruktúrované výsledky v CSV formáte. Zdrojový kód je dôkladne okomentovaný. Dokumentácia v README.md poskytuje prehľad projektu, zoznam požiadaviek, inštrukcie pre inštaláciu a podrobný návod na replikáciu experimentov krok za krokom, od nastavenia prostredia po spustenie testov a následnú evaluáciu výstupov. Takto je zabezpečené, že každý používateľ môže presne zopakovať experimenty a overiť získané výsledky. Žiadne dáta ani kód nepodliehajú dohode o mlčanlivosti (NDA).

2.6 Udržateľnosť a vplyv na životné prostredie

Pre zabezpečenie dlhodobej životaschopnosti a použiteľnosti výsledkov tohto výskumu bola navrhnutá stratégia udržateľnosti, ktorá zahŕňa technické, dokumentačné a environmentálne aspekty. Hlavným cieľom je vytvoriť riešenie, ktoré zostane relevantné a funkčné aj po dokončení štúdia a môže slúžiť ako základ pre budúci výskum či praktické aplikácie.

2.6.1 Technická udržateľnosť

Technický návrh systému bol realizovaný s dôrazom na modularitu a jednoduchú rozšíriteľnosť. Všetky komponenty sú oddelené do samostatných modulov s jasnými rozhraniami, čo umožňuje ich aktualizáciu či rozšírenie bez ovplyvnenia celej architektúry. Použitie štandardizovaných formátov (JSON pre konfiguráciu, CSV pre výsledky) a otvorených knižníc zabezpečuje kompatibilitu s budúcimi technológiami a zároveň umožňuje reprodukovateľnosť experimentov, ako bolo podrobne popísané v predchádzajúcej sekcii 2.5.

Z hľadiska modelov sa práca zameriava na prompt engineering namiesto trénovania vlastných modelov, čo výrazne redukuje potrebu budúcich zásahov. Optimalizačné techniky sú navrhnuté tak, aby neboli viazané na konkrétny model, čo umožňuje ich aplikáciu na novšie verzie LLM bez zásadných úprav. Takto sa minimalizuje environmentálny dopad, pretože nevyžaduje energeticky náročné trénovanie modelov pri každej aktualizácii.

2.6.2 Dokumentácia a archivácia

Kompletná dokumentácia projektu je sprístupnená vo verejnom Git repozitári, ktorý obsahuje podrobné inštrukcie pre inštaláciu a spustenie systému v súbore README.md, komentovaný zdrojový kód vysvetľujúci funkčnosť jednotlivých častí a všetky výsledky experimentov z troch meraní v adresári experiments/. Repozitár je archivovaný na platforme GitHub.

2.6.3 Environmentálna udržateľnosť

Práca aktívne prispieva k zníženiu environmentálnej stopy výskumu v oblasti umelej inteligencie prostredníctvom niekoľkých kľúčových stratégií. Metodológia podporuje používanie menších, lokálne bežiacich jazykových modelov

namiesto závislosti od veľkých cloudových riešení, čím sa eliminujú prenosy dát cez sieť a znižuje sa celková energetická náročnosť. Promptovacie techniky sú navrhnuté s cieľom zníženia spotreby tokenov, čo priamo vedie k úspore výpočtových zdrojov a skracuje čas spracovania.

2.6.4 Strategia rozšírenia

Systém je navrhnutý tak, aby ho bolo možné jednoducho rozširovať bez zásahov do základného kódu. Architektúra systému umožňuje jednoduché pridanie nových úloh (use cases) prostredníctvom rozšírenia konfiguračného JSON súboru bez potreby úpravy základného kódu. Používatelia môžu rozširovať repertoár promptovacích techník jednoduchým pridaním nových záznamov do sekcie base_prompts v config.json. Pre pridanie nových modelov stačí vložiť identifikátor modelu do zoznamu models a stiahnuť ho cez Ollama. Rovnako môže používateľ jednoducho zmeniť premennú EVAL_MODEL v evaluation.py na ľubovoľný dostupný model.

2.7 Zamestnateľnosť

Vďaka tejto bakalárskej práci som si výrazne rozvinul zručnosti v oblasti prompt engineeringu, ktorý sa rýchlo stáva dôležitou kompetenciou na trhu práce. Naučil som sa kombinovať praktické programátorské schopnosti (vývoj v Pythone, prácu s API a automatizáciu testovacích procesov), s porozumením optimalizácie interakcie s veľkými jazykovými modelmi.

Počas práce som získal cenné skúsenosti s návrhom experimentálnych štúdií, analýzou dát a testovaním rôznych prístupov. Tieto zručnosti zodpovedajú požiadavkám, ktoré sa v súčasnosti objavujú na trhu práce, kde sa kladie dôraz

na praktické zručnosti v oblasti generatívnej AI. Štúdia [34] navyše uvádza, že pre pozície vyžadujúce zručnosti v prompt engineeringu často stačí bakalárske vzdelanie a praktické skúsenosti, čo priamo potvrdzuje relevanciu tejto bakalárskej práce pre vstup na trh práce.

Získané skúsenosti mi otvárajú kariérne možnosti v rôznych technických rolách, vrátane pozícií AI Engineer, Prompt Engineer či ML Engineer. Práca tak vytvára pevný základ pre ďalší profesionálny rozvoj v oblasti umelej inteligencie.

2.8 Teamwork, diverzita a inklúzia

V rámci projektového tímu zloženého z IT experta, lingvistu, etika a doménového špecialistu by som navrhol jasne usporiadaný spôsob spolupráce s využitím agilného projektového riadenia (Scrum). Rozdelenie rolí by bolo nasledovné: IT expert by zodpovedal za implementáciu testovacieho systému a API komunikáciu, lingvista by pomáhal formulovať prompty a hodnotiť jazykovú kvalitu výstupov, etik by posudzoval bezpečnostné riziká a navrhoval etické odporúčania, a doménový špecialista by definoval relevantné use cases a overoval praktickú použiteľnosť výsledkov. Základom komunikácie by boli pravidelné týždenné stretnutia (online alebo prezenčné, trvajúce 30-60 minút), na ktorých by sa dôležité témy vysvetľovali zrozumiteľným jazykom. Pre plynulý chod by sme použili jednoduchý nástroj na riadenie úloh (napr. Trello), kde by mal každý jasne určené úlohy a termíny. Pri nezhodách by sme dospeli ku riešeniu spoločnou diskusiou, pričom by sme uprednostnili argumenty podložené experimentálnymi dátami a etickými princípmi.

Dôležitou súčasťou by bola podpora rôznorodosti a otvorenej komunikácie. To znamená, že každý člen tímu by mal priestor vyjadriť svoj názor spôsobom, ktorý mu vyhovuje, a rozhodnutia by sa prijímali spoločne v atmosfére vzá-

jomného rešpektu. Aby sme predišli nedorozumeniam spôsobeným rozdielnou terminológiou, vytvorili by sme spoločný slovník pojmov a pravidelne (napr. na začiatku každého stretnutia) by sme kontrolovali priebeh práce. V súlade s výskumom v oblasti AI a diverzity [35] by zahrnutie rôznorodých pohľadov pomohlo identifikovať a eliminovať skryté predpojatosti vo výsledných systémoch. Takýto prístup by nielen uľahčil splnenie cieľov projektu, ale viedol by aj k vytvoreniu kvalitnejšieho a inkluzívnejšieho riešenia.

Kapitola 3

Záver

Táto bakalárska práca sa zameriava na problém, ktorý je v oblasti umelej inteligencie čoraz aktuálnejší: absenciu jednotného a systematického prístupu k tvorbe kvalitných promptov. Cieľom práce bolo vytvoriť teoretický základ pre efektívne využívanie veľkých jazykových modelov a navrhnúť metodiku, ktorá by umožnila systematické porovnávanie rôznych promptovacích techník. Celá práca je rozdelená na dve hlavné časti: konceptuálnu a vedeckú. Táto kapitola uzatvára konceptuálnu časť, zatiaľ čo vedecká časť prezentuje samotné merania, výsledky a ich analýzu. Vedecká časť práce bola odovzdaná ako samostatný dokument pod názvom `BP_VCP_AndrejKristof.pdf`.

3.1 Osobný odborný rozvoj a získané poznatky

Práca predstavuje významný krok v mojom odbornom rozvoji v oblasti prompt engineeringu. Venoval som sa štúdiu existujúcich prístupov, analýze literatúry a návrhu metodiky na hodnotenie správania jazykových modelov. Súčasťou riešenia je testovací systém v Pythone na automatizované testovanie modelov

(kvalita, rýchlosť, tokeny) a evaluačný skript využívajúci externý LLM na objektívne hodnotenie výstupov. Podrobné výsledky experimentov sú prezentované vo vedeckej časti práce.

3.2 Prínos pre prax a používateľov

Práca vytvorila východisko pre vznik praktickej príručky zameranej na prompt engineering, ktorá bude slúžiť ako pomôcka pre široké spektrum používateľov. Používateľ vďaka príručke ušetrí čas pri tvorbe promptov a dosiahne kvalitnejšie výstupy s nižšími nákladmi na tokeny. Výstupy práce sú smerované tak, aby boli využiteľné nielen pre technicky zdatných používateľov, ale aj pre širšiu verejnosť, ktorá chce pracovať s umelou inteligenciou efektívnejšie a cielenejšie. Testovací systém a evaluačný skript sú zároveň praktickými nástrojmi, ktoré možno nasadiť v tímoch pracujúcich s LLM na automatizáciu testovania promptov, čo zvyšuje efektivitu a konzistentnosť.

3.3 Širší význam a zhrnutie

Z dlhodobého hľadiska práca prispieva k zvýšeniu zrozumiteľnosti a dostupnosti umelej inteligencie. Poukazuje na dôležitosť jasného a zodpovedného používania umelej inteligencie a otvára priestor pre ďalší výskum v oblasti práce s jazykovými modelmi. Práca zároveň ukazuje, ako môže prompt engineering prepojiť informatiku s etikou (bezpečnostné riziká, jailbreaking, etické odporúčania).

Výsledky tejto práce, doplnené o experimentálne overenie v druhej časti, predstavujú ucelený teoretický a praktický základ pre efektívne využívanie veľkých jazykových modelov s presahom do vzdelávania, výskumu aj praxe.

Zoznam skratiek

Skratka	Význam
AI	Umelá inteligencia (Artificial Intelligence)
API	Aplikačné programovacie rozhranie (Application Programming Interface)
BPO	Optimalizácia promptov v čiernej skrinke (Black-Box Prompt Optimization)
CoT	Reťazové uvažovanie (Chain-of-Thought)
CSV	Hodnoty oddelené čiarkou (Comma-Separated Values)
JSON	JavaScript Object Notation (formát výmeny dát)
LLM	Veľký jazykový model (Large Language Model)
ML	Strojové učenie (Machine Learning)
MLM	Maskovaný jazykový model (Masked Language Model)
PCA	Analýza hlavných komponentov (Principal Component Analysis)
SLM	Malý jazykový model (Small Language Model)
ToT	Stromové uvažovanie (Tree-of-Thoughts)

Zoznam obrázkov

2.1 Sekvenčný diagram procesu testovania jazykových modelov . . . 11
2.2 Ilustrácia Tree of Thoughts 13
2.3 Príklad výstupu v súbore full_outputs.txt 19

Literatúra

[1] Jesse G. Meyer, Ryan J. Urbanowicz, Patrick C.N. Martin et al. "ChatGPT and large language models in academia: opportunities and challenges". In: *BioData Mining* 16.20 (2023). DOI: 10.1186/s13040-023-00339-9. URL: https://link.springer.com/article/10.1186/s13040-023-00339-9.

[2] Banghao Chen et al. "Unleashing the potential of prompt engineering for large language models". In: *Patterns* 6.6 (2025), s. 101260. ISSN: 2666-3899. DOI: https://doi.org/10.1016/j.patter.2025.101260. URL: https://www.sciencedirect.com/science/article/pii/S2666389925001084.

[3] Pranab Sahoo et al. *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*. 2025. arXiv: 2402.07927 [cs.AI]. URL: https://arxiv.org/abs/2402.07927.

[4] Satvik Tripathi et al. "A Hitchhiker's Guide to Good Prompting Practices for Large Language Models in Radiology". In: *Journal of the American College of Radiology* 22.7 (2025). Practice in Community, Rural, and Safety Net Settings, s. 841–847. ISSN: 1546-1440. DOI: https://doi.org/10.1016/j.jacr.2025.02.051. URL: https://www.sciencedirect.com/science/article/pii/S1546144025001565.

[5] Jaekeol Choi. "Efficient Prompt Optimization for Relevance Evaluation via LLM-Based Confusion Matrix Feedback". In: *Applied Sciences* 15.9 (2025).

ISSN: 2076-3417. DOI: 10.3390/app15095198. URL: https://www.mdpi.com/2076-3417/15/9/5198.

[6] Lechen Zhang et al. *SPRIG: Improving Large Language Model Performance by System Prompt Optimization*. 2024. arXiv: 2410.14826 [cs.CL]. URL: https://arxiv.org/abs/2410.14826.

[7] Flavio Corradini, Matteo Leonesi a Marco Piangerelli. "State of the Art and Future Directions of Small Language Models: A Systematic Review". In: *Big Data and Cognitive Computing* 9.7 (2025). ISSN: 2504-2289. DOI: 10.3390/bdcc9070189. URL: https://www.mdpi.com/2504-2289/9/7/189.

[8] Jiale Cheng et al. "Black-Box Prompt Optimization: Aligning Large Language Models without Model Training". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. Lun-Wei Ku, Andre Martins a Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, aug. 2024, s. 3201–3219. DOI: 10.18653/v1/2024.acl-long.176. URL: https://aclanthology.org/2024.acl-long.176/.

[9] Claudio Spiess et al. *AutoPDL: Automatic Prompt Optimization for LLM Agents*. 2025. arXiv: 2504.04365 [cs.LG]. URL: https://arxiv.org/abs/2504.04365.

[10] Wenwu Li et al. *A Survey of Automatic Prompt Engineering: An Optimization Perspective*. 2025. arXiv: 2502.11560 [cs.AI]. URL: https://arxiv.org/abs/2502.11560.

[11] Masaru Yamada. "Optimizing Machine Translation through Prompt Engineering: An Investigation into ChatGPT's Customizability". In: *Proceedings of Machine Translation Summit XIX, Vol. 2: Users Track*. Ed. Masaru Yamada a Felix do Carmo. Macau SAR, China: Asia-Pacific Association for Ma-

chine Translation, sept. 2023, s. 195–204. URL: https://aclanthology.org/2023.mtsummit-users.19/.

[12] Antonio Sabbatella et al. "Prompt Optimization in Large Language Models". In: *Mathematics* 12.6 (2024). ISSN: 2227-7390. DOI: 10.3390/math12060929. URL: https://www.mdpi.com/2227-7390/12/6/929.

[13] Mateusz Bystroński et al. *LatentPrompt: Optimizing Promts in Latent Space*. 2025. arXiv: 2508.02452 [cs.CL]. URL: https://arxiv.org/abs/2508.02452.

[14] Ashutosh Kumar et al. *The Ethics of Interaction: Mitigating Security Threats in LLMs*. 2024. arXiv: 2401.12273 [cs.CR]. URL: https://arxiv.org/abs/2401.12273.

[15] Zijian Feng et al. "PromptExplainer: Explaining Language Models through Prompt-based Learning". In: *Findings of the Association for Computational Linguistics: EACL 2024*. Ed. Yvette Graham a Matthew Purver. St. Julian's, Malta: Association for Computational Linguistics, mar. 2024. URL: https://aclanthology.org/2024.findings-eacl.60/.

[16] Andrew Gao. *Prompt Engineering for Large Language Models*. 2023. URL: https://ssrn.com/abstract=4504303.

[17] Takeshi Kojima et al. "Large Language Models are Zero-Shot Reasoners". In: *Advances in Neural Information Processing Systems*. Ed. S. Koyejo et al. Zv. 35. Curran Associates, Inc., 2022, s. 22199–22213. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/8bb0d291acd4acf06ef112Paper-Conference.pdf.

[18] Tom Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Ed. H. Larochelle et al. Zv. 33. Curran Associates, Inc., 2020, s. 1877–1901. URL: https://proceedings.

neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

[19] Lei Wang et al. "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. Anna Rogers, Jordan Boyd-Graber a Naoaki Okazaki. Toronto, Canada: Association for Computational Linguistics, júl 2023, s. 2609–2634. DOI: 10.18653/v1/2023.acl-long.147. URL: https://aclanthology.org/2023.acl-long.147/.

[20] Aobo Kong et al. "Better Zero-Shot Reasoning with Role-Play Prompting". In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Ed. Kevin Duh, Helena Gomez a Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, jún 2024, s. 4099–4113. DOI: 10.18653/v1/2024.naacl-long.228. URL: https://aclanthology.org/2024.naacl-long.228/.

[21] Xuezhi Wang et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. 2023. arXiv: 2203.11171 [cs.CL]. URL: https://arxiv.org/abs/2203.11171.

[22] Jiacheng Liu et al. "Generated Knowledge Prompting for Commonsense Reasoning". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. Smaranda Muresan, Preslav Nakov a Aline Villavicencio. Dublin, Ireland: Association for Computational Linguistics, máj 2022, s. 3154–3169. DOI: 10.18653/v1/2022.acl-long.225. URL: https://aclanthology.org/2022.acl-long.225/.

[23] Zekun Li et al. "Guiding Large Language Models via Directional Stimulus Prompting". In: *Advances in Neural Information Processing Systems*. Ed.

A. Oh et al. Zv. 36. Curran Associates, Inc., 2023, s. 62630–62656. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/c5601d99ed028448f29d1dae2e4a926d-Paper-Conference.pdf.

[24] Shunyu Yao et al. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models". In: *Advances in Neural Information Processing Systems*. Ed. A. Oh et al. Zv. 36. Curran Associates, Inc., 2023, s. 11809–11822. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/271db9922b8d1f4dd7aaef84ed5ac703-Paper-Conference.pdf.

[25] Huaixiu Steven Zheng et al. "Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models". In: *International Conference on Learning Representations*. Ed. B. Kim et al. Zv. 2024. 2024, s. 20279–20316. URL: https://proceedings.iclr.cc/paper_files/paper/2024/file/592da1445a51e54a3987958b5831948f-Paper-Conference.pdf.

[26] Marah Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. 2024. arXiv: 2404.14219 [cs.CL]. URL: https://arxiv.org/abs/2404.14219.

[27] Alexander H. Liu et al. *Ministral 3*. 2026. arXiv: 2601.08584 [cs.CL]. URL: https://arxiv.org/abs/2601.08584.

[28] Qwen et al. *Qwen2.5 Technical Report*. 2025. arXiv: 2412.15115 [cs.CL]. URL: https://arxiv.org/abs/2412.15115.

[29] Imen Azaiz et al. *Open, Small, Rigmarole – Evaluating Llama 3.2 3B's Feedback for Programming Exercises*. 2025. arXiv: 2504.01054 [cs.CY]. URL: https://arxiv.org/abs/2504.01054.

[30] Daya Guo et al. "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning". In: *Nature* 645.8081 (2025), 633–638. ISSN: 1476-

4687. DOI: 10.1038/s41586-025-09422-z. URL: http://dx.doi.org/10.1038/s41586-025-09422-z.

[31] An Yang et al. *Qwen3 Technical Report*. 2025. arXiv: 2505.09388 [cs.CL]. URL: https://arxiv.org/abs/2505.09388.

[32] Gemma Team et al. *Gemma 3 Technical Report*. 2025. arXiv: 2503.19786 [cs.CL]. URL: https://arxiv.org/abs/2503.19786.

[33] Qinyuan Ye et al. "Prompt Engineering a Prompt Engineer". In: *Findings of the Association for Computational Linguistics: ACL 2024*. Ed. Lun-Wei Ku, Andre Martins a Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, aug. 2024, s. 355–385. DOI: 10.18653/v1/2024.findings-acl.21. URL: https://aclanthology.org/2024.findings-acl.21/.

[34] Mahdi Ahmadi, Neda Khosh Kheslat a Adebola Akintomide. *Generative AI Impact on Labor Market: Analyzing ChatGPT's Demand in Job Advertisements*. 2024. arXiv: 2412.07042 [cs.CY]. URL: https://arxiv.org/abs/2412.07042.

[35] Rifat Ara Shams, Didar Zowghi a Muneera Bano. "AI and the quest for diversity and inclusion: a systematic literature review". In: *AI and Ethics* 5.1 (2025), s. 411–438. ISSN: 2730-5961. DOI: 10.1007/s43681-023-00362-w. URL: https://doi.org/10.1007/s43681-023-00362-w.